

EuroShop — SQL Portfolio Project

I built this while completing the Microsoft SQL Foundations course on Coursera. The course used a music store database for exercises, which was fine for learning syntax — but I wanted to work on something that felt closer to what an analyst actually does at a company. So I built this alongside the course.

EuroShop is a made-up European e-commerce business. Sneakers, jackets, sunglasses, bags — four markets across Germany, UK, Italy, and France. I designed the database from scratch, inserted realistic data with patterns I could actually find through SQL, and wrote business questions I thought a real analyst would be asked.

The database

Five tables. 330 rows.

categories — 8 product categories

products — 20 products across those categories

customers — 50 customers across DE, GB, IT, FR

orders — 100 orders from January to July 2024

order_items — 150 line items linking orders to products

How they connect:

categories

|

category_id

|

products — product_id — order_items — order_id — orders

|

customer_id

|

customers

Files

euroshop_01_create_tables.sql — schema with constraints and foreign keys

euroshop_02_insert_data.sql — data insert

euroshop_03_select_queries.sql — 11 business questions answered in SQL

euroshop_04_views.sql — 4 reusable views

The 11 business questions

I tried to write questions a business analyst would actually get asked, not just "select all from table."

Q1 — Which product sold the most units?

Q2 — Which product made the most money?

- Q3** — Which product do people return the most?
- Q4** — Which country is our biggest market?
- Q5** — How has revenue trended month by month?
- Q6** — How many customers came back and ordered again?
- Q7** — How are people paying — card, PayPal, or debit?
- Q8** — Which products are people buying together?
- Q9** — Who are the top customers and are they above or below average spend?
- Q10** — Which products have a return rate above their category average?
- Q11** — Which country topped revenue each month, and did it go up or down from the month before?
-

What I actually found in the data

- Ultra Boost 22 outsold everything — 22 units, nearly 50% more than second place
 - The Leather Jacket generated strong revenue but had a 42.9% return rate — for every 7 orders, 3 came back
 - Germany was the top market every single month, January through July
 - 33 of 50 customers placed more than one order — 66% retention
 - Payment methods were nearly even: credit card 41%, PayPal 30%, debit 29%
-

SQL concepts this covers

I did not set out to cover concepts — I set out to answer the questions. These are what the questions required:

- CREATE TABLE with primary keys, foreign keys, CHECK constraints
- INSERT, TRUNCATE CASCADE
- SELECT, WHERE, ORDER BY, LIMIT
- GROUP BY, HAVING
- INNER JOIN across up to four tables
- Self-join (Q8 — product pairs)

- Aliases, dot notation
 - Aggregate functions — SUM, COUNT, AVG, ROUND
 - CASE WHEN inside COUNT for conditional counting
 - Date functions — TO_CHAR for month grouping
 - String functions — CONCAT for full names
 - Window functions — OVER(), PARTITION BY, RANK(), LAG()
 - CTEs — WITH...AS blocks for multi-step logic
 - Views — CREATE VIEW, querying views, filtering on aliases
-

The 4 views

view_revenue_by_country -- which market is biggest

view_monthly_revenue -- revenue trend over time

view_return_rates -- product return rate breakdown

view_customer_retention -- customers who ordered more than once

How to run it

You need PostgreSQL installed. I used VS Code with the PostgreSQL extension.

Run the files in order:

1. euroshop_01_create_tables.sql

2. euroshop_02_insert_data.sql

3. euroshop_03_select_queries.sql

4. euroshop_04_views.sql

Why I built this

The Microsoft SQL Foundations course was good for getting the concepts. But finishing a course and being able to do something useful with SQL are not the same thing. I wanted to close that gap before applying for analyst roles.

This is not a perfect project. Some queries were hard to write and I had to work through them carefully. But every query in this repo runs, returns real results, and answers a question I actually wanted to know the answer to.

Built with PostgreSQL · VS Code · Microsoft SQL Foundations (Coursera)